Zora

PRODUCT REQUIREMENT DOCUMENT (PRD)

Version: 1.0.0
Status: Approved

TARGET STACK	AI ENGINE	DESIGN SYNC	DOCUMENT TYPE
React + FastAPI	Gemma 3:1b (Ollama)	Stitch MCP + Cursor	PRD & Tech Architecture

1. Executive Summary & Vision

Zora is a specialized developer tool designed to bridge the gap between learning complex technical concepts (e.g., P2P networking, consensus mechanisms, memory management) and sharing key insights publicly. Developers and students often lack the time to draft formatted content for different platforms after intensive learning sessions. Zora automates this workflow by ingesting raw technical notes and instantly generating tailored posts for LinkedIn, Medium, and Substack using a locally hosted, privacy-focused LLM (Gemma 3:1b via Ollama).

2. User Personas & Use Cases

Persona A: The Tech Student

Goal: Build a public proof-of-work portfolio on LinkedIn while taking online courses or bootcamp tracks.
Pain Point: Limited time to re-write rough notes into engaging posts after long studying hours.
Solution: Ingest rough takeaways and generate concise LinkedIn updates with optional code snippets and hashtags.

Persona B: The Software Engineer

Goal: Maintain a technical blog on Medium or Substack documenting architecture patterns.
Pain Point: Formatting long-form articles, adding intro hooks, and structuring markdown takes hours.
Solution: Generate structured markdown article drafts complete with headings, technical breakdowns, and summaries.

3. Feature Scope & Functional Requirements

Feature	Priority	Description	Acceptance Criteria
Platform Selector	P0 - Core	Toggle target output between LinkedIn, Medium, and Substack.	UI adjusts layout constraints, word length limits, and formatting style.
Context Ingestion	P0 - Core	Textarea for raw study notes, code snippets, or topic bullet points.	Supports up to 4,000 characters of input context with real-time word counter.
Local AI Generation	P0 - Core	FastAPI orchestration sending structured prompts to local Ollama instance.	Sub-3-second streaming/generation latency using Gemma 3:1b model.
Live Preview & Edit	P1 - High	Side-by-side markdown preview with platform-specific character counters.	Displays rendered markdown, code highlight blocks, and quick-copy action.

4. Platform-Specific Output Guidelines

Zora enforces strict styling rules based on the user's selected publishing platform to ensure maximum engagement and compatibility without requiring manual user edits:

Target Platform	Length / Limits	Formatting Style	Tone & Call-to-Actions
LinkedIn	150 – 300 words (<3,000 chars)	Short paragraphs, bullet lists, emojis, trailing hashtag block.	Professional, energetic, key takeaways, ends with an engagement question.
Medium	600 – 1,200 words	Full Markdown: H2/H3 headers, sub-sections, formatted code blocks.	In-depth technical explanation, ELI5 analogies, architectural breakdown.
Substack	400 – 800 words	Newsletter format, warm introduction, structured key points, conclusion.	Personal journey tone, conversational, developer-to-developer insight.

5. Non-Functional Requirements & Performance Targets

< 3.5s
Generation Latency
Gemma 3:1b local execution threshold

100%
Data Privacy
Zero cloud telemetry; fully local execution

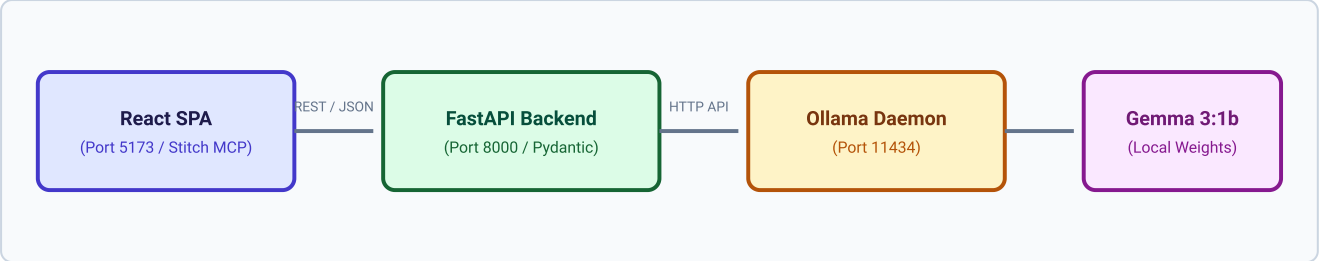
0 KB
API Costs
Free tier local inference via Ollama

6. User Experience & Interface Requirements

- **Single-Page Workspace:** Split-view layout (Input Studio left 45%, Output Preview right 55%). Zero page navigation required during generation workflow.
- **Instant Platform Switching:** Switching tabs re-formats or prompts quick regeneration without losing user context inputs.
- **One-Click Actionability:** Global "Copy to Clipboard" with instant visual feedback badge ("Copied!"), plus raw Markdown export (`.md`).
- **Refinement Shortcuts:** Quick action chips below output preview ("Make concise", "Add code example", "Simplify tone") that send instant delta refinement requests.

7. Technical System Architecture

Zora operates entirely on a local developer environment. The React SPA communicates with a lightweight FastAPI gateway, which handles prompt construction, validation, and streaming connection to the Ollama daemon running `gemma3:1b`.



8. API Interface Schema (FastAPI)

The backend exposes a clean endpoint `/api/v1/generate` accepting structured context and platform directives.

```
# Post Request Payload Schema
POST /api/v1/generate
Content-Type: application/json

{
  "platform": "linkedin",              // Options: "linkedin" | "medium" | "substack"
  "topic": "P2P Blockchain Networks",  // Primary subject line
  "context": "Studied peer discovery, gossip protocols, and NAT traversal...",
  "tone": "technical",                 // Options: "technical" | "storytelling" | "casual" |
"eli5"
  "include_hashtags": true,            // Optional boolean
  "include_code": true                 // Optional boolean
}
```

9. Ollama & Gemma 3:1b Prompt Engineering Pipeline

Because `gemma3:1b` is a lightweight model, FastAPI builds highly explicit system prompts to enforce formatting constraints, avoiding hallucination and preventing meta-commentary preamble.

```
SYSTEM_PROMPT = """You are Zora, an expert technical writer for developers.
Format the output strictly for platform: {platform}.
Rules:
- Directly output the final post content. Do NOT include greetings, preamble, or meta
explanations.
- Use tone: {tone}.
- If platform is 'linkedin': Keep under 250 words, use concise bullet points and direct
takeaways.
- Include code snippets: {include_code}.
- Include relevant hashtags: {include_hashtags}."""
```

10. Cursor MCP & Stitch Integration Workflow

Development is accelerated using **Cursor IDE** integrated with **Stitch UI Engine** via Model Context Protocol (MCP). This enables direct code generation aligned with design primitives.

Cursor + Stitch MCP Execution Flow

1. **Design Sync:** Cursor fetches Stitch layout tokens, component hierarchies, and CSS color variables directly via MCP.
2. **Component Binding:** UI state changes in React (e.g., `platform`, `tone`) trigger auto-generated FastAPI client hooks.
3. **Local Model Verification:** Cursor runs local integration test suites invoking Ollama health checks before serving UI state.

11. Project Directory Structure

```
zora-app/
├── backend/
│   ├── app/
│   │   ├── main.py              # FastAPI application entrypoint & CORS config
│   │   ├── services/
│   │   │   └── ollama_client.py # Async HTTP client for Ollama API (gemma3:1b)
│   │   ├── schemas/
│   │   │   └── request.py       # Pydantic models for validation
│   │   └── prompts/
│   │       └── templates.py     # Platform-specific prompt generators
│   └── requirements.txt         # fastapi, uvicorn, httpx, pydantic
└── frontend/
    ├── src/
    │   ├── components/          # InputStudio.jsx, PreviewPanel.jsx, PlatformTabs.jsx
    │   ├── hooks/               # useGenerateContent.js
    │   └── App.jsx              # Main Single-Page Application view
    └── package.json             # React 18, Tailwind CSS / Lucide icons
```

12. Local Setup & Deployment Verification

1. Start Local Model (Ollama)

```
# Pull and run Gemma 3:1b
ollama pull gemma3:1b
ollama run gemma3:1b
```

2. Launch FastAPI Server

```
cd backend
pip install -r requirements.txt
uvicorn app.main:app --reload
```

Development Milestone Checklist

✔ Ollama daemon accessible at `http://localhost:11434`
✔ FastAPI Swagger documentation passing tests at `http://localhost:8000/docs`
✔ React SPA rendering Stitch design tokens with live response streaming.